

Predlog projekta iz predmeta Sistemi bazirani na znanju

Uroš Radukić SV54/2022

Opis problema

Motivacija

Trenutno razvijam video igru za [Nordeus Challenge](#). U pitanju je strateška igra po potezima (turn-based RPG), gde igrač i neprijatelj naizmenično biraju kako će iskoristiti svoj potez, dok ne ostane samo jedan od njih živ. Jedna od stavki u izazovu koja je poželjna je da neprijateljski AI ne bira nasumično kako će iskoristiti svoj potez, već da na smislen način odabere koji mu je najbolji potez. Veštačka inteligencija neprijatelja u video igrama je jedan od ključnih faktora koji utiču na kvalitet iskustva igrača. To je posebno izraženo u žanru strateških igara po potezima, ponašanje neprijatelja direktno utiče na to koliko je igra zanimljiva, i koliki izazov predstavlja. Tu sam uvideo idealnu priliku da iskoristim Drools i znanje stečeno na predmetu Sistemi bazirani na znanju i uz svoje ekspertsko znanje iz oblasti strateških video igara dizajniram AI za neprijatelje.

Pregled problema

Postojeća rešenja i njihova ograničenja

Konačni automati (FSM) dugo su bili standard u video igrama. Svako stanje definiše ponašanje, a tranzicije reaguju na događaje. Prednost je jednostavnost implementacije, ali mana je što nema nikakvu fleksibilnost, neprijatelj ima fiksni broj stanja i ne može da reaguje na kombinacije uslova koje dizajner nije eksplicitno predvideo. Konkretna primena u turn-based žanru dokumentovana je u radu [Application of the Finite State Machine Method in the Desktop-Based "Heroes Of Dawn" RPG Turn-Based Game](#) (Riyanto et al., 2022), gde FSM upravlja izborom napada neprijatelja na osnovu trenutnog HP-a i mane. Sistem funkcioniše, ali svaki novi tip neprijatelja zahteva ručno pisanje novih stanja i tranzicija.

Stabla ponašanja (Behavior Trees) nude hijerarhijsku kompoziciju akcija i uslova, što ih čini fleksibilnijim od FSM-a. Popularizovana su igrom *Halo 2* (Bungie, 2004), gde je Bungie prvi put u industriji upotrebio BT za upravljanje borbenom AI NPC-eva. Detalji su opisani u GDC prezentaciji [Handling Complexity in the Halo 2 AI](#) (Isla, GDC 2005). Mana je što su i dalje ručno projektovana i skaliranje postaje problem. Istraživanje [Practical and Theoretical Issues of Evolving Behaviour Trees for a Turn-based Game](#) (Lim et al., 2019) pokazuje da BT za turn-based igre brzo postaju nepregledni i teški za održavanje čim broj akcija poraste.

Pretraga stabla odluka - Monte Carlo Tree Search (MCTS) je algoritam koji simulira hiljade slučajnih ishoda iz trenutnog stanja igre kako bi pronašao statistički najperspektivniji potez, bez potrebe za ručno kodiranim pravilima. Osnova je položena u radu [Monte-Carlo Tree Search: A New Framework for Game AI](#) (Chaslot et al., AAAI 2008), a algoritam je postigao vrhunske rezultate u igrama poput Shogija i Go-a. Ograničenje je visoka računarska cena po potezu i loša primenljivost na igre sa velikim brojem stanja ili skrivenim informacijama, kao i to što ne postoji čitljivo obrazloženje zašto je određeni potez izabran.

Utility AI dodeljuje numeričke vrednosti mogućim akcijama na osnovu konteksta i bira onu sa najvišim skorom. Najpoznatija primena je serija *The Sims*, gde svaki objekat u svetu objavljuje skup akcija sa procenjenom korisnošću. Arhitektura je detaljno opisana u radu [Artificial Intelligence in The Sims Series](#) (Bourse, INRIA 2014). Mada fleksibilan, sistem postaje nepredvidiv pri većem broju faktora i teško ga je ispratiti. Promena jedne krive korisnosti može da promeni ponašanje na nepredviđene načine u sasvim drugom delu igre.

Mašinsko učenje i neuronske mreže nude adaptivnost, ali zahtevaju ogromne količine trening podataka i visoku računarsku snagu. Referentni primer je *AlphaStar* (DeepMind, 2019) za StarCraft II, opisan u radu [Grandmaster level in StarCraft II using multi-agent reinforcement learning](#), (Vinyals et al., *Nature* 2019). Sistem je dostigao nivo grandmastera, ali je zahtevao trening na milionima partija i potpuno je neproziran: nemoguće je pročitati zašto je AI izabrao konkretan potez, što nije prihvatljivo za igru gde je konzistentnost i predvidivost ponašanja neprijatelja od suštinskog značaja.

Naše rešenje i prednosti

Ideja projekta je da se AI neprijatelja implementira korišćenjem **Drools rule engine-a**. Ovaj pristup donosi prednosti:

- **Eksplcitna pravila** čitljiva kao poslovne politike. Ovo je posebno značajno u razvoju video igara zbog toga što omogućava dizajnerima da ručno unose pravila sa minimalnim znanjem programiranja. To olakšava ceo proces iterativnog razvoja jer se gubi potreba za prisustvom programera pri svakom testiranju novih parametara.
- **CEP (Complex Event Processing)** za detekciju obrazaca tokom borbenih događaja korišćenjem klizajućih prozora (sliding windows). Umesto da posmatramo konkretno vremensku odredbu, posmatraćemo prednoinh N poteza i gledati da donesemo neki zaključak. Jedna od dobrih osobina neprijateljskog AI bila bi da se i adaptira na konkretno, specifično ponašanje igrača, na primer ukoliko je igrač u poslednjih nekoliko poteza samo koristio fizičke napade, možemo zaključiti da povećavanje odbrane ima veći značaj nego inače.
- **Backward chaining** za ciljno zaključivanje, poput toga "Kojim potezom/kombinacijom poteza bi mogao da ubije igrača". Na osnovu konkretnog cilja gleda kako može da ga ispuni.
- **Višeslojnu arhitekturu** (L1 percepcija → L2 taktika → L3 akcija) koja razdvaja odgovornosti i nudi fleksibilnost pri biranju konkretnog poteza
- **Arhetipovi** korišćenjem Template-a. Kako bi se bolje izrazile razlike u neprijateljima i njihove karakterne osobine (a bez dodatne komplikacije koja bi nasledila u ostalim pristupima), možemo koristiti template za podesavanje razlicitih pragova za činjenice. Na primer, za činjenicu `HEALTH_CRITICAL` , drugačiji prag bismo imali za goblina koji je plasiljiv (recimo 40%) i za viteza koji je neustrasiv (10%).

Metodologija rada

Ulazi u sistem (Input)

Za svaki potez neprijatelja sistem prima sledeće **ulazne činjenice** koje se upisuju u Drools radnu memoriju:

Činjenica	Polja	Sadržaj
EnemyFact	id, archetype, maxHp, hpPercent, currentStamina, maxStamina, currentMana, maxMana, attack, defense, magic	Kompletno stanje neprijatelja, uključujući arhitip i sve statistike
PlayerFact	currentHp, maxHp, attack, defense, magic	Kompletno stanje igrača
MoveOption	moveId, moveCategory, costType, costValue, projectedValue	Jedan zapis po svakom potez koji neprijatelj može priuštiti; projectedValue je unapred izračunata procenjena šteta
ActiveStatusEffect	target (entityId), statType, value	Aktivan status efekat na igraču ili neprijatelju (negativna vrednost = debuff)

CombatEventFact	type, source, target, moveCategory, amount, turnTimestamp	Događaj iz istorije borbe, unet u CEP entry-point "combat-stream" kao @role(event)
-----------------	---	--

Izvedene činjenice koje sistem sam generiše primenom pravila:

Činjenica	Generiše je	Moguće vrednosti	Opis
CantFinishPlayer	ConfirmNoImmediateKill (L1)	marker (bez polja)	Insertuje se kada nijedna MoveOption ne može ubiti igrača ovaj potez; pokretač za sve L1 i CEP perpcijske regule
ResourceStatus	L1 pravila	type=STARVED, resourceName="stamina" ili "mana"	Resurs neprijatelja ispod 20% maksimuma
StatComparison	L1 pravila	PLAYER_PHYSICAL_DOMINATES, PLAYER_MAGIC_DOMINATES	Statistička prednost igrača nad neprijateljem
BurstDamageAssessment	CEP pravilo	severity=HIGH, total (ukupna šteta)	Igrač je naneo > 35% neprijateljevog maxHp u poslednjih 3 događaja
PlayerBehaviorProfile	CEP pravila	PHYSICAL_SPAMMER, COMBO_PLAYER	Detektovani obrazac ponašanja igrača
PerceivedThreat	L1 pravila	level=CRITICAL (< 25% HP), HIGH (25–50% HP), LOW (≥ 50% HP)	Procenjeni nivo ugroženosti neprijatelja na osnovu trenutnog HP-a; uvek se generiše tačno jedna vrednost
Tactic	L2 pravila (salience 1–80)	MAXIMIZE_DAMAGE, SELF_BUFF, DEBUFF_PLAYER, CONSERVATIVE, DRAIN	Izabrana borbena taktika za tekući potez
EnemyDecision	L3 pravila / fallback	moveId, priority, reason	Izlaz sistema — odluka o potezu sa prioritetom i tekstualnim obrazloženjem

Izlazi iz sistema (Output)

Sistem proizvodi jednu odluku po pozivu:

- **EnemyDecision** : identifikator poteza koji neprijatelj treba da odigra, sa prioritetom i tekstualnim obrazloženjem

Iz skupa svih generisanih odluka bira se ona sa najvišim prioritetom i vraća `CombatService` -u koji je izvršava.

Baza znanja

Popunjavanje baze znanja: Pravila su statički definisana u DRL fajlovima i učitavaju se pri pokretanju aplikacije. Konfiguracija likova, poteza i predmeta čita se iz JSON fajlova (`characters.json` , `moves.json` , `items.json`) — dodavanje novog neprijatelja ili poteza ne zahteva izmenu Java koda, već samo dodavanje u JSON fajl.

Kompletna lista pravila po DRL fajlovima:

`level1-perception.drl` — **L1 Percepcija**

Pravilo	Uslov	Akcija
ConfirmNoImmediateKill	Nijedna MoveOption(projectedValue >= playerHp) ne postoji	INSERT CantFinishPlayer()
EvaluateEnemyThreatCritical	CantFinishPlayer() + hpPercent < 0.25	INSERT PerceivedThreat(CRITICAL)
EvaluateEnemyThreatHigh	CantFinishPlayer() + 0.25 ≤ hpPercent < 0.50	INSERT PerceivedThreat(HIGH)
EvaluateEnemyThreatLow	CantFinishPlayer() + hpPercent ≥ 0.50	INSERT PerceivedThreat(LOW)
EvaluateResourceStaminaStarved	CantFinishPlayer() + stamina < 20% maksimuma	INSERT ResourceStatus(STARVED, "stamina")
EvaluateResourceManaStarved	CantFinishPlayer() + mana < 20% maksimuma	INSERT ResourceStatus(STARVED, "mana")
DetectStatDisadvantage	CantFinishPlayer() + player.attack – enemy.defense > 5	INSERT StatComparison(PLAYER_PHYSICAL_DOMINATES)
DetectMagicThreat	CantFinishPlayer() + player.magic > enemy.magic × 1.3	INSERT StatComparison(PLAYER_MAGIC_DOMINATES)

`accumulate-burst.drl` — **CEP akumulacija**

Pravilo	Uslov	Akcija
AssessBurstDamageRisk	CantFinishPlayer() + Zbir player DAMAGE_DEALT u poslednjih 3 događaja > 35% neprijateljevog maxHp	INSERT BurstDamageAssessment(total, HIGH)

`cep-patterns.drl` — **CEP obrasci ponašanja**

Pravilo	Uslov	Akcija
DetectPlayerPhysicalSpammer	CantFinishPlayer() + Igrač koristio DAMAGE_PHYSICAL ≥ 3 puta u poslednjih 4 MOVE_USED događaja (window:length(4))	INSERT PlayerBehaviorProfile(PHYSICAL_SPAMMER)

DetectPlayerComboSetup	CantFinishPlayer() + Igrač koristio ENEMY_DEBUFF pa odmah zatim napad (after[0,4])	INSERT PlayerBehaviorProfile(COMBO_PLAYER)
------------------------	--	---

backward-queries.dr1 — Queryji

Pomoćni queryji (direktan pattern match)

Ovi queryji su obični Drools predikati nad radnom memorijom — jednoslojna disjunkcija pattern-a, bez rekurzije i bez generisanja međucijeva. Drools ih razrešava jednim prolazom kroz činjenice, pa nisu *backward chaining* u užem smislu (graf ciljeva ne postoji), iako se sintaksno pozivaju isto, sa `?` prefiksom.

Upit	Parametri	Dokazuje
canKillPlayer	playerHp	Postoji MoveOption čiji projectedValue >= playerHp
isVulnerableTo	entityId, damageType	Za damageType == "physical": entitet ima aktivan defense debuff (ActiveStatusEffect) Ili urođeno nisku odbranu (defense <= 4). Tri OR grane razlikuju izvor podatka: status efekat, PlayerFact ili EnemyFact.

Backward chaining — canKillPlayerInNTurns

Upit	Parametri
canKillPlayerInNTurns	playerHp, enemyMana, enemyStamina, turnsLeft

Postavlja cilj ("dokazati da neprijatelj može ubiti igrača za N poteza") i onda za svaki potez koji nanosi štetu, napravi "scenario" kakva bi situacija bila sa resursima i enemy helathom, i onda rekurzivno poziva sa tim novim stanjem sa N-1 poteza: `?canKillPlayerInNTurns( playerHp - $dmg, regenClamp(enemyMana - manaCost($costType, $cost) + $mag, $maxM), regenClamp(enemyStamina - staminaCost($costType, $cost) + $att, $maxS), turnsLeft - 1; )`.

Struktura query-ja:

```
query "canKillPlayerInNTurns"(int playerHp, int enemyMana, int enemyStamina, int turnsLeft)
    // Bazni slučaj: igrač već poražen
    eval(playerHp <= 0)
or
    // Rekurzivni: postoji damaging potez M koji možemo priuštiti,
    // i nakon njegove primene (+ regen za sledeći potez) cilj je dostižan u N-1 poteza
    eval(turnsLeft > 0 && playerHp > 0)
    $e : EnemyFact($mag : magic, $att : attack, $maxM : maxMana, $maxS : maxStamina)
    $m : MoveOption($dmg : projectedValue, $cost : costValue, $costType : costType,
                    projectedValue > 0)
    eval(canAfford($costType, $cost, enemyMana, enemyStamina))
    ?canKillPlayerInNTurns(
        playerHp - $dmg,
        regenClamp(enemyMana - manaCost($costType, $cost) + $mag, $maxM),
        regenClamp(enemyStamina - staminaCost($costType, $cost) + $att, $maxS),
        turnsLeft - 1; )
end
```

Kako Drools razrešava ovaj cilj:

1. **Bazni slučaj** — Drools prvo proverava prvu OR granu: ako je `playerHp <= 0` (igrač već mrtav), cilj je dokazan i query vraća uspeh.

2. **Rekurzivni slučaj** — ako bazni nije ispunjen, Drools razrešava drugu OR granu:

- Pronalazi `EnemyFact` (jedinствен, daje konstante: magic, attack, maxMana, maxStamina)
- Iterira kroz sve `MoveOption` činjenice koje su damaging (`projectedValue > 0`) i affordable (`canAfford` helper)
- **Za svaki takav potez** Drools formira novi cilj sa transformisanim stanjem: `playerHp - dmg` , ažurirane resursi posle troška + regena za sledeći potez (`regenClamp` osigurava nenegativnost i ne prelazak max-a), `turnsLeft - 1`
- Rekurzivno poziva `?canKillPlayerInNTurns` sa novim ciljem
- **Ako bilo koja grana stigne do baznog slučaja**, cela rekurzija uspeva i query vraća dokaz

Pomoćne funkcije (definisane u istom DRL fajlu):

Funkcija	Svrha
<code>canAfford(t, c, m, s)</code>	true ako enemy ima dovoljno resursa za potez tipa t cene c
<code>manaCost(t, c)</code>	Vraća c ako je t == "mana", inače 0 (selektivno trošenje)
<code>staminaCost(t, c)</code>	Vraća c ako je t == "stamina", inače 0
<code>regenClamp(v, max)</code>	<code>Math.max(0, Math.min(max, v))</code> — klampuje resurs nakon regena

Šta čini ovo pravim backward chaining-om:

- **Cilj se ne dokazuje direktno**, već se dekomponuje na potcijeve (svaki potez = novi međucilj)
- **Generiše se proof tree**: koren je početni cilj, grananje je po svakom potezu, listovi su bazni slučajevi
- **Pretraga je depth-first sa unifikacijom parametara** — Drools razrešava jednu po jednu granu, vraća se nazad ako grana ne uspe
- **Drools sam upravlja rekurzijom** — nema iterativne kontrole iz Jave; samo definisanje queryja u DRL-u opisuje *šta* je dokaz, ne *kako* tražiti

Konkretna primer rekurzivnog razvijanja:

Za GoblinMage sa mana=45, magic=9; player HP=60; potezi `firebolt` (15 mane, 20 dmg) i `arcaneSurge` (20 mane, 25 dmg) — poziv `?canKillPlayerInNTurns(60, 45, 20, 3)` :

```
?canKill(60, 45, 20, 3)
├ Grana firebolt: ?canKill(40, 39, 24, 2)           // 45-15+9=39 mane za sledeći potez
| └ Grana firebolt: ?canKill(20, 33, 28, 1)
|   └ Grana firebolt: ?canKill(0, 27, 32, 0) → eval(playerHp<=0) ✓ DOKAZ
|     └ ...
|   └ ...
└ Grana arcaneSurge: ?canKill(35, 34, 24, 2)
  └ ...
```

Pošto je dovoljno da jedna grana stigne do baznog slučaja, čim Drools nađe putanju `firebolt → firebolt → firebolt` koja zadovoljava, query vraća uspeh i ostatak stabla se ne istražuje.

Upotreba u sistemu — koristi se na dva mesta u `level2-tactics.drl` :

- ChooseFinishSoonTactic (salience 60): pri PerceivedThreat(LOW) poziva ? canKillPlayerInNTurns(\$hp, \$em, \$es, 3;) — ako postoji 3-turn kill plan, prelazi u MAXIMIZE_DAMAGE
- ChooseConservativeTactic (salience 45): poziva not ?canKillPlayerInNTurns(...;) kao guard — blokira ulaz u konzervativni mod ako postoji put do pobeđe

level12-tactics.dr1 — L2 Selekcija taktike

Pravilo	Salience	Uslov	Akcija
ChooseDrainOnHighBurst	80	BurstDamageAssessment(HIGH) + DRAIN potez dostupan	INSERT Tactic(DRAIN)
ChooseDebuffOnHighBurst	79	BurstDamageAssessment(HIGH) + ENEMY_DEBUFF potez dostupan	INSERT Tactic(DEBUFF_PLAYER)
ChooseSelfBuffOnHighBurst	78	BurstDamageAssessment(HIGH) + SELF_BUFF potez dostupan	INSERT Tactic(SELF_BUFF)
ChooseConservativeOnHighBurst	77	BurstDamageAssessment(HIGH), nijedna druga opcija nije dostupna	INSERT Tactic(CONSERVATIVE)
ChooseFinishSoonTactic	60	PerceivedThreat(LOW) + ? canKillPlayerInNTurns(playerHp, enemyMana, enemyStamina, 3) uspešan	INSERT Tactic(MAXIMIZE_DAMAGE)
ChooseDebuffPlayerTactic	50	StatComparison(PLAYER_PHYSICAL_DOMINATES ili PLAYER_MAGIC_DOMINATES) + ENEMY_DEBUFF dostupan	INSERT Tactic(DEBUFF_PLAYER)
ChooseConservativeTactic	45	ResourceStatus(STARVED) + not ? canKillPlayerInNTurns(playerHp, enemyMana, enemyStamina, 3) (blokirano kad postoji kill plan)	INSERT Tactic(CONSERVATIVE)
ChooseDrainTactic	40	PerceivedThreat(CRITICAL ili HIGH) + DRAIN dostupan	INSERT Tactic(DRAIN)
ChooseSelfBuffTactic	35	PerceivedThreat(LOW ili HIGH) + SELF_BUFF dostupan	INSERT Tactic(SELF_BUFF)

level13-action.dr1 — L3 Selekcija poteza

Pravilo	Uslov	Akcija
EmitImmediateKillMove	?canKillPlayer(playerHp) uspešan (direktno iz ulaznih činjenica; uzajamno isključivo sa ConfirmNoImmediateKill)	INSERT EnemyDecision(letalni potez, prioritet 100)
EmitMaximizeDamagePhysicalVulnerable	Tactic(MAXIMIZE_DAMAGE) + ? isVulnerableTo("player", "physical") + not EnemyDecision(priority >= 10)	INSERT EnemyDecision(best physical move, prioritet 10)

EmitMaximizeDamageMagicVulnerable	Tactic(MAXIMIZE_DAMAGE) + ? isVulnerableTo("player", "magical") + not EnemyDecision(priority >= 10)	INSERT EnemyDecision(best magic move, prioritet 10)
EmitMaximizeDamageAny	Tactic(MAXIMIZE_DAMAGE) + not EnemyDecision(priority >= 9), bez detektovane ranjivosti	INSERT EnemyDecision(best damage move, prioritet 9)
EmitSelfBuffMove	Tactic(SELF_BUFF) + not EnemyDecision(priority >= 7)	INSERT EnemyDecision(self- buff potez, prioritet 7)
EmitDebuffMove	Tactic(DEBUFF_PLAYER) + not EnemyDecision(priority >= 7)	INSERT EnemyDecision(debuff potez, prioritet 7)
EmitDrainMove	Tactic(DRAIN) + not EnemyDecision(priority >= 7)	INSERT EnemyDecision(drain potez, prioritet 7)
EmitConservativeMove	Tactic(CONSERVATIVE) + not EnemyDecision(priority >= 6)	INSERT EnemyDecision(najjeftiniji potez, prioritet 6)

fallback.dr1 — Fallback pravila

Pravilo	Salience	Uslov	Akcija
ChooseFallbackTactic	1 (L2)	CantFinishPlayer() + nijedna taktika nije izabrana	INSERT Tactic(MAXIMIZE_DAMAGE)
EmitFallbackDecision	—	Tactic() postoji + nijedna odluka nije generisana	INSERT EnemyDecision(najjeftiniji dostupni potez, prioritet 1)

Mehanizam forward chaining-a — Java kod samo upisuje ulazne činjenice i poziva `session.fireAllRules()`. Ne postoji nikakvo eksplicitno određivanje redosleda faza. Kaskada između slojeva nastaje isključivo kroz zavisnosti podataka, sa jasnom rasviljicom na samom početku:

```
Ulazne činjenice (EnemyFact, PlayerFact, MoveOption, ...)  
├── postoji MoveOption koja može ubiti igrača?  
│   DA → EmitImmediateKillMove → INSERT EnemyDecision ← KRAJ  
│  
└── NE → ConfirmNoImmediateKill → INSERT CantFinishPlayer  
    ↓ okida sve L1 i CEP regule (čiste zavisnosti podataka)  
    L1 percepcija INSERT: PerceivedThreat, ResourceStatus, StatComparison  
    CEP pravila  INSERT: BurstDamageAssessment, PlayerBehaviorProfile  
    ↓ nove činjenice okidaju L2 pravila  
    L2 taktika   INSERT: Tactic ← jedini sloj sa salience  
    ↓ Tactic okida L3 pravila  
    L3 akcija    INSERT: EnemyDecision ← KRAJ
```

Svaki sloj je aktiviran **činjenicama koje je prethodni sloj upisao**.

- `EmitImmediateKillMove` i `ConfirmNoImmediateKill` su **uzajamno isključivi** čistim zavisnostima podataka; kada ubistvo postoji, uslov `not MoveOption(projectedValue >= playerHp)` u `ConfirmNoImmediateKill` nikad nije ispunjen; kada ne postoji, `query ?canKillPlayer` u `EmitImmediateKillMove` ne nalazi dokaz
- L1 i CEP pravila nemaju salience — svako upisuje drugačiju činjenicu i ne takmiče se međusobno; redosled njihovog paljenja ne utiče na ishod
- L2 burst pravila (salience 77–80) koriste salience za prioritet: `ChooseDrainOnHighBurst` je poželjniji od `ChooseSelfBuffOnHighBurst`; `not Tactic()` guard deaktivira ostala L2 pravila čim jedno uspe
- L3 pravila nemaju salience — priority polje u samom `EnemyDecision` faktu vrši selekciju; `not EnemyDecision(priority >= N)` guard sprečava prepisivanje odluke višeg prioriteta; `bestDecision()` na kraju bira onu sa najvišim prioritetom

Template-i i arhetipovi

Različiti tipovi neprijatelja moraju da reaguju različito na istu situaciju: vitez od 30% HP ne bi trebalo da paniči kao goblin od 30% HP. Umesto da pišemo zasebna pravila za svaki arhetip (što duplira logiku) ili da jednim globalnim pragom `HEALTH_CRITICAL` ukinemo karakterizaciju, koristimo **programatski generisana pravila po arhetipu**: jedna struktura pravila se parametrizuje različitim vrednostima pragova. Implementacija je u `DroolsConfig.generateArchetypeDrl()` — Java metoda koja pri pokretanju aplikacije konstruiše `template-archetypes.drl` na osnovu niza `ARCHETYPES`, dodaje ga u `Drools KieFileSystem` zajedno sa statičkim DRL-ovima, i build prolazi normalno kroz `KieBuilder`.

Podržani arhetipovi i njihovi pragovi (definisano u `DroolsConfig.ARCHETYPES`):

Arhetip	criticalHpPct	retreatHpPct	aggressionHpPct	Karakter
PHYSICAL_BRAWLER	0.33	0.25	0.50	Tipičan ratnik — paniči srednje rano
MAGIC_CASTER	0.30	0.30	0.45	Krhki čarobnjak — ne juri agresivno
MAGIC_DRAINER	0.35	0.35	0.50	Drain specijalista — oprezno bira borbe
PHYSICAL_SKIRMISHER	0.30	0.20	0.55	Brz i preduzimljiv — agresivan dok god ima HP
BALANCED_TANK	0.25	0.15	0.35	Izdržljiv — kasno paniči, kreće u napad i kad je povređen

Mapiranje konkretnih neprijatelja na arhetipove drži `DroolsEnemyAIService.ARCHETYPE_MAP`: na primer, `goblinWarrior / minotaur / centaur` su `PHYSICAL_BRAWLER`, `witch / archdemon` su `MAGIC_DRAINER`, `dragon / skeletonArcher` su `BALANCED_TANK`. Više različitih neprijatelja deli isti arhetip i time istu logiku donošenja odluka, ali se vizuelno i mehanički razlikuju u ostatku igre.

Pravila koja se generišu po arhetipu: za svaki red u tabeli iznad `generateArchetypeDrl()` proizvodi dva pravila sa identičnom strukturom, samo se konstante razlikuju:

Generisano pravilo	Salience	Uslov	Akcija
<code>EvaluateHealthCritical_<ARCHETYPE></code>	100 (L1)	<code>EnemyFact(archetype == <ARCHETYPE>, hpPercent <</code>	<code>INSERT PerceivedThreat(CRITICAL)</code>

		criticalHpPct) + not PerceivedThreat()	
ChooseAggressiveTactic_<ARCHETYPE>	50 (L2)	EnemyFact(archetype == <ARCHETYPE>, hpPercent >= aggressionHpPct) + not PerceivedThreat(CRITICAL) + not Tactic()	INSERT Tactic(MAXIMIZE_DAMAGE)

Konkretno, za `MAGIC_CASTER` arhetip generiše se (sa `criticalHpPct = 0.30` i `aggressionHpPct = 0.45`):

```
rule "EvaluateHealthCritical_MAGIC_CASTER"
    salience 100
when
    EnemyFact(archetype == Archetype.MAGIC_CASTER, hpPercent < 0.30)
    not PerceivedThreat()
then
    insert(new PerceivedThreat(ThreatLevel.CRITICAL));
end

rule "ChooseAggressiveTactic_MAGIC_CASTER"
    salience 50
when
    EnemyFact(archetype == Archetype.MAGIC_CASTER, hpPercent >= 0.45)
    not PerceivedThreat(level == ThreatLevel.CRITICAL)
    not Tactic()
then
    insert(new Tactic(TacticType.MAXIMIZE_DAMAGE));
end
```

Ista struktura, samo različite konstante (`Archetype.<X>` , `criticalHpPct` , `aggressionHpPct`), ponavlja se za svih 5 arhetipova — što na izlazu daje 10 dodatnih pravila u baze znanja.

Interakcija sa univerzalnim L1 pravilima: statički `EvaluateEnemyThreatCritical/High/Low` iz `level1-perception.dr1` koriste fiksne pragove (0.25 / 0.50) i okidani su `CantFinishPlayer()`. Template-ovana `EvaluateHealthCritical_<X>` pravila imaju **viši salience (100)** i koriste arhetip-specifičan prag — kada se obojica mogu primeniti, template-ovano pravilo pali prvo i `not PerceivedThreat()` guard sprečava univerzalno pravilo da prepíše rezultat. Tako arhetip dobija prioritet, ali ako njegov prag nije dosegnut, sistem se vraća na univerzalnu logiku kao bezbedan default.

Prednost u praksi: dodavanje novog arhetipa (npr. `BERSERKER` koji nikad ne paniči i agresivno napada: `criticalHpPct = 0.10` , `aggressionHpPct = 0.20`) zahteva:

1. dodavanje enum vrednosti `BERSERKER` u `Archetype.java`
2. dodavanje jednog reda u `DroolsConfig.ARCHETYPES`
3. (opcionalno) mapiranje konkretnog neprijatelja u `ARCHETYPE_MAP`

Bez pisanja novih DRL pravila, bez izmene postojeće logike, bez ponavljanja koda. Template-i ovde služe i kao **mehanizam za izražavanje karakternosti neprijatelja** — ista borbena situacija proizvodi različite percepcije i različite taktike u zavisnosti od toga ko je u njoj, što je suštinski element kvalitetne AI u strateškim igrama.

Šta se template-uje, a šta ne: nije sve parametrizovano po arhetipu. Pragove vezane za HP (kritičnost, agresivnost) ima smisla razlikovati — to je deo karaktera neprijatelja. Ali generička percepcija kao što je „mana je pala ispod 20%“ ili „igrač fizički dominira“ je univerzalna i živi u statičkim L1 pravilima (`level1-perception.dr1`). Template-i pokrivaju samo onu dimenziju gde arhetip donosi razliku.

Konkretna primer rezonovanja

Scenario

Neprijatelj: GoblinMage, nivo 2 (arhitip: `MAGIC_CASTER`)

- HP: 30 / 70 (43%) | Mana: 42 / 45 | Stamina: 3 / 20
- Statistike: napad=4, odbrana=3, magija=9
- Dostupni potezi:
 - `firebolt` : `DAMAGE_MAGIC`, cena: 15 mane
 - `arcaneSurge` : `DAMAGE_MAGIC`, cena: 20 mane
 - `hexShield` : `SELF_BUFF`, cena: 10 mane
 - `manaDrain` : `DRAIN`, cena: 10 stamine → **nije dostupan** (stamina=3 < 10)
- Nema aktivnih statusnih efekata

Igrač: Knight, nivo 2

- HP: 60 / 120 (50%) | Statistike: napad=12, odbrana=10, magija=4
- Nema aktivnih statusnih efekata

Istorija borbe (poslednja 4 događaja na `combat-stream`):

Redosled	Akter	Potez	Kategorija	Šteta
1	Igrač	<code>slash</code>	<code>DAMAGE_PHYSICAL</code>	12
2	Igrač	<code>slash</code>	<code>DAMAGE_PHYSICAL</code>	14
3	Igrač	<code>battleCry</code>	<code>SELF_BUFF</code>	—
4	Igrač	<code>slash</code>	<code>DAMAGE_PHYSICAL</code>	11

Korak 1 — Provera ubojstva: `ConfirmNoImmediateKill`

Pravila `ConfirmNoImmediateKill` i `EmitImmediateKillMove` pale direktno iz ulaznih činjenica i uzajamno su isključiva, samo jedno od njih može da pali u datom potezu.

`ConfirmNoImmediateKill` proverava: postoji li `MoveOption` čiji `projectedValue` $\geq$ `player.currentHp` ?

Potez	Projektivna šteta	Player currentHp	Može ubiti?
<code>firebolt</code>	20	60	Ne, 20 < 60
<code>arcaneSurge</code>	25	60	Ne, 25 < 60
<code>hexShield</code>	— (nije šteta)	60	Ne

Nijedna `MoveOption` ne zadovoljava uslov → `ConfirmNoImmediateKill` pali → **INSERT**
`CantFinishPlayer()` ← okidač za sve L1 i CEP regule

Da je igrač imao, recimo, 22 HP, `arcaneSurge` bi zadovoljio uslov: `ConfirmNoImmediateKill` ne bi palilo, već `EmitImmediateKillMove` — i odmah bi bila emitovana odluka sa prioritetom 100, bez prolaska kroz L2 i L3.

Korak 2 — L1 Percepcija: okidač `CantFinishPlayer`

Upisom `CantFinishPlayer()` u koraku 1, sva L1 pravila postaju aktivna.

EvaluateEnemyThreatHigh :

$hpPercent = 30/70 = 0.43 \rightarrow 0.25 \leq 0.43 < 0.50 \checkmark \rightarrow$ **INSERT** `PerceivedThreat(HIGH)` ← okidač za *drain/self-buff* taktike

EvaluateResourceStaminaStarved :

$stamina\ 3/20 = 15\% < 20\% \rightarrow$ **INSERT** `ResourceStatus(STARVED, "stamina")`

EvaluateResourceManaStarved :

$mana\ 42/45 = 93\% > 20\% \rightarrow$ uslov nije ispunjen

DetectStatDisadvantage :

$player.attack(12) - enemy.defense(3) = 9 > 5 \rightarrow$ **INSERT** `StatComparison(PLAYER_PHYSICAL_DOMINATES)`

Korak 3 — CEP nad event stream-om: okidač `CantFinishPlayer`

CEP pravila takođe zahtevaju `CantFinishPlayer()` i procesiraju tok borbenih događaja korišćenjem klizajućih prozora.

AssessBurstDamageRisk (accumulate sum, window: poslednje 3 štete):

$sum\ DAMAGE_DEALT\ od\ igrača:\ 12 + 14 + 11 = 37\ 37 > 70 \times 0.35 = 24.5 \checkmark \rightarrow$ **INSERT**
`BurstDamageAssessment(37, HIGH)`

DetectPlayerPhysicalSpammer (accumulate count, window: poslednja 4 poteza):

$DAMAGE_PHYSICAL\ potezi\ igrača:\ slash,\ slash,\ battleCry\ (ne),\ slash = 3 \geq 3 \checkmark \rightarrow$ **INSERT**
`PLayerBehaviorProfile(PHYSICAL_SPAMMER)`

Korak 4 — L2 Selekcija taktike: aktivirana `BurstDamageAssessment` činjenicom

Upisom `BurstDamageAssessment(37, HIGH)` u koraku 2, burst pravila postaju aktivna. Sa salienceom 77–80 — višim od ostalih L2 pravila — evaluiraju se prva.

ChooseDrainOnHighBurst (salience 80):

$BurstDamageAssessment(HIGH) \checkmark \mid MoveOption(DRAIN)\ dostupan? \rightarrow$ `manaDrain` nije dostupan
($stamina=3 < 10$) $\rightarrow$ uslov nije ispunjen

ChooseDebuffOnHighBurst (salience 79):

$BurstDamageAssessment(HIGH) \checkmark \mid MoveOption(ENEMY_DEBUFF)\ dostupan? \rightarrow$ `GoblinMage` nema debuff
poteze $\rightarrow$ uslov nije ispunjen

ChooseSelfBuffOnHighBurst (salience 78):

```
BurstDamageAssessment(HIGH) ✓ | MoveOption(SELF_BUFF) dostupan? → hexShield je SELF_BUFF ✓ |  
not Tactic() ✓ → INSERT Tactic(SELF_BUFF)
```

Preostala L2 pravila:

- ChooseDebuffPlayerTactic (50): StatComparison ✓ ali nema ENEMY_DEBUFF → ne pali
- ChooseConservativeTactic (45): ResourceStatus(STARVED) ✓, ali not Tactic() **ne važi** → ne pali

Korak 5 — L3 Selekcija poteza: aktivirana Tactic činjenicom

Upisom Tactic(SELF_BUFF) , pravilo EmitSelfBuffMove postaje aktivno.

EmitSelfBuffMove :

```
Tactic(SELF_BUFF) ✓ | not EnemyDecision(priority >= 7) ✓ MoveOption(SELF_BUFF) =  
hexShield → INSERT EnemyDecision("hexShield", 7, "Self-buff tactic: hexShield")
```

Ishod i obrazloženje

GoblinMage odabira hexShield , buff-uje se umesto da napada.

Odluka je nastala kroz čisti forward chaining — svaki korak je bio automatski okidan novim činjenicama:

1. PlayerFact + MoveOption → ConfirmNoImmediateKill pali → kill nije moguć → INSERT CantFinishPlayer
2. CantFinishPlayer → L1 pali → INSERT PerceivedThreat(HIGH) , ResourceStatus(STARVED) , StatComparison
3. CantFinishPlayer → CEP pali → INSERT BurstDamageAssessment(HIGH) , PlayerBehaviorProfile(PHYSICAL_SPAMMER)
4. BurstDamageAssessment(HIGH) → ChooseSelfBuffOnHighBurst pali → INSERT Tactic(SELF_BUFF)
5. Tactic(SELF_BUFF) → EmitSelfBuffMove pali → INSERT EnemyDecision("hexShield", 7, ...)

Primećujemo da ChooseConservativeTactic (salience 45) **nije** palilo, iako je ResourceStatus(STARVED) bio prisutan — burst pravilo (salience 78) je ubacilo Tactic pre nego što je konzervativno pravilo imalo prilike. not Tactic() guard je potom automatski deaktivirao sva preostala L2 pravila.

Ceo proces, od sirovih statistika do konkretnog poteza, odvija se transparentno kroz čitljiva pravila, sa jasnim tragom zaključivanja koji je moguće ispitati i promeniti u svakom koraku.